

实验六：链式工具调用

实验六：链式工具调用

实验目的

1. 理解链式工具调用的概念和应用场景
2. 掌握链式调用的实现方法
3. 学习工具调用上下文管理
4. 实现复杂任务的多步骤处理

实验内容

实验过程

1. 代码结构分析 本实验包含两个 Python 文件：

chat_client.py - 链式工具调用客户端 - `execute_tool()`: 执行工具调用 - `ChainedCallContext`: 链式调用上下文管理类 - `build_analysis_prompt()`: 构建分析提示词 - `parse_llm_response()`: 解析 LLM 响应 - `execute_chained_tool_call()`: 执行链式工具调用

test_chained_calls.py - 测试脚本 - 测试链式调用上下文 - 测试提示词构建 - 测试文件搜索和操作

2. 链式调用上下文管理 上下文类设计

```
class ChainedCallContext:
    def __init__(self, max_iterations=10):
        self.max_iterations = max_iterations
        self.steps = []
        self.variables = {}
        self.current_iteration = 0

    def add_step(self, tool_name, arguments, result):
        self.steps.append({
            "tool_name": tool_name,
```

```
        "arguments": arguments,
        "result": result
    })
    self.current_iteration += 1

def set_variable(self, key, value):
    self.variables[key] = value

def get_variable(self, key, default=None):
    return self.variables.get(key, default)
```

可用工具列表

```
def get_tools_config():
    return [
        {
            "type": "function",
            "function": {
                "name": "list_directory",
                "description": "列出目录文件",
                "parameters": {"dir_path": {"type": "string"}}
            }
        },
        {
            "type": "function",
            "function": {
                "name": "read_file",
                "description": "读取文件内容",
                "parameters": {"dir_path": {"type": "string"}, "file_name": {"type": "string"}}
            }
        },
        {
            "type": "function",
            "function": {
                "name": "create_file",
                "description": "创建文件",
                "parameters": {"dir_path": {"type": "string"}, "file_name": {"type": "string"},
            }
        },
        {
            "type": "function",
            "function": {
                "name": "curl",
```

```

        "description": "访问网页",
        "parameters": {"url": {"type": "string"}}
    }
}
]

```

3. 链式调用流程 执行流程

```

def execute_chained_tool_call(user_request, max_iterations=10):
    context = ChainedCallContext(max_iterations=max_iterations)

    for iteration in range(max_iterations):
        # 构建分析提示词
        analysis_prompt = build_analysis_prompt(
            user_request,
            context.get_steps_history(),
            context.variables
        )

        # 调用 LLM 获取下一步决策
        messages = [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": analysis_prompt}
        ]

        response = call_llm_non_stream(messages)

        # 解析响应
        done, result = parse_llm_response(response)

        if done:
            return result

        # 执行工具调用
        if isinstance(result, dict) and "name" in result:
            tool_name = result.get("name")
            tool_arguments = result.get("arguments", {})

            tool_result = execute_tool(tool_name, tool_arguments)
            context.add_step(tool_name, tool_arguments, tool_result)

            # 设置变量供后续步骤使用
            if tool_name == "list_directory":

```

```
files = json.loads(tool_result)
file_list = [f["名称"] for f in files if f.get("类型") == "文件"]
context.set_variable("file_list", file_list)
```

响应解析

```
def parse_llm_response(response):
    content = response.get("content", "")
    tool_calls = response.get("tool_calls", None)

    # 检查 tool_calls 格式
    if tool_calls and isinstance(tool_calls, list) and len(tool_calls) > 0:
        tool_call = tool_calls[0]
        tool_name = tool_call.get('function', {}).get('name')
        arguments = tool_call.get('function', {}).get('arguments', {})
        return False, {"name": tool_name, "arguments": arguments}

    # 检查 JSON 格式响应
    if content:
        try:
            decision = json.loads(content)
            if decision.get("done") is True:
                answer = decision.get("answer", "")
                return True, answer
            if decision.get("done") is False:
                tool_call = decision.get("tool_call", {})
                return False, tool_call
        except json.JSONDecodeError:
            pass
```

实验结果

功能验证

功能	chat_client.py	test_chained_calls.py
目录列表	正确	正确
文件读取	正确	正确
文件创建	正确	正确
网页访问	正确	正确
链式调用执行	正确	正确
上下文管理	正确	正确

输出示例

=====

开始链式工具调用

用户请求：读取 1.txt 和 2.txt 两个文件，把两个数相加的和写入 result.txt

最大迭代次数：10

=====

[迭代 1/10]

[调试] 当前消息数量：2

[调试] 调用LLM...

[解析成功] done=false, 调用工具：list_directory

决定调用工具：list_directory

参数：{"dir_path": "practice06"}

工具返回：[{"名称": "1.txt", "类型": "文件", ...}, {"名称": "2.txt", "类型": "文件", ...}]

[迭代 2/10]

[解析成功] done=false, 调用工具：read_file

决定调用工具：read_file

参数：{"dir_path": "practice06", "file_name": "1.txt"}

工具返回：42

[迭代 3/10]

[解析成功] done=false, 调用工具：read_file

决定调用工具：read_file

参数：{"dir_path": "practice06", "file_name": "2.txt"}

工具返回：58

[迭代 4/10]

[解析成功] done=false, 调用工具：create_file

决定调用工具：create_file

参数：{"dir_path": "practice06", "file_name": "result.txt", "content": "100"}

工具返回：成功：文件 result.txt 已创建并写入内容

[迭代 5/10]

[解析成功] done=true, 回答：已成功完成任务！读取了1.txt(42)和2.txt(58)，计算和为100，并写入result.txt文件。

[完成] 任务完成

最终回答：已成功完成任务！读取了1.txt(42)和2.txt(58)，计算和为100，并写入result.txt文件。

实验总结

实验成果

1. 实现了链式工具调用框架，支持多步骤任务
2. 实现了上下文管理，支持变量传递
3. 实现了灵活的工具调用格式（tool_calls 和 JSON 格式）
4. 实现了任务完成检测和迭代控制

技术要点

1. **状态管理**: 通过 ChainedCallContext 维护调用状态和变量
2. **迭代控制**: 限制最大迭代次数，防止无限循环
3. **响应解析**: 支持多种响应格式（tool_calls、JSON、文本）
4. **变量传递**: 工具执行结果可作为变量供后续步骤使用

改进建议

1. 可添加任务规划功能，自动分解复杂任务
2. 可实现错误重试机制
3. 可添加工具调用优先级
4. 可实现并行工具调用